

dualview.c

[illegible]

```

39 #define B_SOLID      0          // 0000 0000 0000 0000
40 #define B_DOTTED     0x1000     // 0001 0000 0000 0000
41 #define B_DASHED     0x2000     // 0010 0000 0000 0000
42 #define STYLE_MASK   0x3000     // 0011 0000 0000 0000
43 //
44 const char *colors[8] = {"black", "red", "green", "yellow", "blue", "magenta", "cyan", "white"};
45 // 索引          0      1      2      3      4      5      6      7
46 // 对应的正是颜色值      000      001      010      011      100      101      110      111
47
48 struct box_props {
49     bool opaque          : 1;
50     unsigned int fill_color : 3;
51     unsigned int          : 4;
52     bool show_border     : 1;
53     unsigned int border_color : 3;
54     unsigned int border_style : 2;
55     unsigned int          : 2;
56 };
57 // 这个结构在内存中的位模式如下
58 // 内存地址位          高位
59 //
60 // 二进制位          15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
61 // 对应的值          2      2      1      3      1
62 // 对应值含义          DASHED GREEN true YELLOW true
63 //
64 //
65 //
66 //
67 // unused 2
68 // border_style
69 // border_color
70 // show_border
71 // unused 4
72 // fill_color
73 // opaque
74
75 union Views { /* 把数据看作结构或unsigned short类型的变量 */
76     struct box_props st_view; // box_props 结构体占2字节
77     unsigned short us_view; // short 类型变量也是占2字节
78 };
79 // union 是联合体，共享内存意思，也就是 us_view 和 st_view 地址相同，因此读取 us_view，也就是读 st_view 的数据

```

从右到左，在第13到14位，在后面的代码注释中，重点理解掩码的作用：任何数，跟掩码取反后与运算，该数掩码位都变成0，也就是清0的作用

加起来正好16位，占2字节

低位，最右边算起，1表示不透明，0表示透明

填充颜色，占3位，看颜色二进制值

未命名结构体成员，填充 4 位，下图用 x 表示

是否显示边框，1显示，0不显示

边框颜色，占3位，看颜色二进制值

边框样式，占2位，

高位，在最左边，看边框线样式的二进制值

低位(内存起始地址，由于us_view和此结构体共用内存，所以us_view的值，也就是这个起始位置所指向的值，也就是1)

```

80 // unsigned short us_view 正好是二字节，占16位，跟 box_props 的位相同，所以可以把数据看作结构类型变量
81
82 void show_settings(const struct box_props *pb);
83 void show_settings1(unsigned short);
84 char *itobs(int n, char *ps);
85 void show_bstr (const char * str);
86
87 int main(void) {
88     /* 创建Views联合，并初始化 initialize struct box view */
89     union Views box = {{true, YELLOW, true, GREEN, DASHED}};
90     //          1      3      1      2      2
91     //          1    011    1    010    10
92     // 注意：未命名的结构位，起到填充占位的作用，没有实际值，用 x 显示代替
93     // 初始化上面的 box 后，内存中二进制位模式如上图所示，由于 Views 是一个共享内存的联合体，所以 us_view 的内存地址，跟 st_view 是一样的
94     // 因此us_view的值，就是内存起始位置第一个值，也就是1（但如果box_props所有成员类型和us_view类型一样，就是另外情况，请看 test.c ）
95     char bin_str[8 * sizeof(unsigned int) + 1];
96     printf("ORIGINAL BOX SETTINGS:\n");
97     show_settings(&box.st_view);
98     printf("\nBox settings using unsigned int view:\n");
99     show_settings1(box.us_view);
100    itobs(box.us_view, bin_str);
101    printf("bits are          : ");
102    show_bstr(bin_str);
103    printf("\n\nn=====\\n");
104    // box.us_view = 1 = 0001 // 上面已经解释了为什么运行到这里这个值为1
105    box.us_view &= ~FILL_MASK; /* 把表示填充色的位清0，如代码31行所描述的一样 */
106    // FILL_MASK = 0xE = 1110
107    // ~0xE = 0001 // 取反就是1为0，0为1
108    // us_view = 0001
109    // &= 0001 = 1
110    box.us_view |= FILL_CYAN; /* 重置填充色， 或运算可以理解为赋上值的意思，因为0或1就等1*/
111    // FILL_BLUE = 0x8 = 1000
112    // FILL_GREEN = 0x4 = 0100
113    // 0x8 | 0x4 = 1100
114    // us_view = 0001
115    // |= 1101 = 13
116    box.us_view ^= OPAQUE; /* 切换是否透明的位，异或运算就是找出不同的地方，只要不同就为1，只要相同就为0 */
117    // OPAQUE = 0x1 = 0001
118    // us_view = 1101
119    // ^= 1100 = 12
120    box.us_view |= BORDER_RED; /* 设置边框为红色，上面说了，或运算其实就是赋值的意思，对着二进制位，仔细思考和理解 */

```

```

121 // BORDER_RED = 0x200 = 0010 0000 0000
122 //          us_view = 0000 0000 1100
123 //          |= 0010 0000 1100 = 524 // 或就是有一个为1就是1
124 box.us_view &= ~STYLE_MASK; /* 把样式的位清0，如代码42行所描述的一样 */
125 // STYLE_MASK = 0x3000 = 0011 0000 0000 0000
126 //          ~0x3000 = 1100 1111 1111 1111
127 //          us_view = 0000 0010 0000 1100
128 //          &= 0000 0010 0000 1100 = 524
129 box.us_view |= B_DOTTED; /* 把样式设置为点 */
130 // B_DOTTED = 0x1000 = 0001 0000 0000 0000
131 //          us_view = 0000 0010 0000 1100
132 //          |= 0001 0010 0000 1100 = 4620
133 printf("\nMODIFIED BOX SETTINGS:\n");
134 show_settings(&box.st_view);
135 printf("\nBox settings using unsigned int view:\n");
136 show_settings1(box.us_view);
137 itobs(box.us_view, bin_str);
138 printf("bits are          : ");
139 show_bstr(bin_str);
140 printf("\n");
141 getchar();
142 return 0;
143 }
144
145 void show_settings(const struct box_props *pb) {
146     printf("Box is          : %s\n", pb->opaque == true ? "opaque" : "transparent");
147     printf("The fill color is    : %s\n", colors[pb->fill_color]);
148     printf("Border              : %s\n", pb->show_border == true ? "shown" : "not shown");
149     printf("The border color is  : %s\n", colors[pb->border_color]);
150     printf("The border style is : ");
151     switch (pb->border_style) {
152     case SOLID:
153         printf("solid\n");
154         break;
155     case DOTTED:
156         printf("dotted\n");
157         break;
158     case DASHED:
159         printf("dashed\n");
160         break;
161     default:

```

```

162     printf("unknown type\n");
163 }
164 }
165
166 void show_settings1(unsigned short us) {
167     printf("box is          : %s\n", (us & OPAQUE) == OPAQUE ? "opaque" : "transparent");
168     printf("The fill color is : %s\n", colors[(us >> 1) & 07]);
169     // 这里重点理解, 为什么右移 1 位, 因为 fill_color 就是 2 到 4 位, 把最右边 1 位的 opaque 移出去,
170     // 然后因为 fill_color 占 3 位, 所以和 07 与运算, 即和 111 进行与运算, 得到的值, 正是颜色的下标(索引), 然后通过下标得到颜色名
171     printf("Border          : %s\n", (us & BORDER) == BORDER ? "shown" : "not shown");
172     printf("The border style is : ");
173     switch (us & STYLE_MASK) {
174     case B_SOLID:
175         printf("solid\n");
176         break;
177     case B_DOTTED:
178         printf("dotted\n");
179         break;
180     case B_DASHED:
181         printf("dashed\n");
182         break;
183     default:
184         printf("unknown type\n");
185     }
186     printf("The border color is : %s\n", colors[(us >> 9) & 07]); // 07 是八进制书写方式, 前面加0
187     // 这里重点理解, 为什么右移 9 位, 因为 border_color 就是 10 到 12 位, 把最右边 9 位移出去, 就剩下含 border_color 的值了
188     // 然后因为 border_color 占 3 位, 所以和 07 与运算, 即和 111 进行与运算, 得到的值, 正是颜色的下标(索引), 然后通过下标得到颜色名
189 }
190
191 char *itobs(int n, char *ps) {
192     int i;
193     const static int size = CHAR_BIT * sizeof(short);
194     for (i = size - 1; i >= 0; i--, n >>= 1)
195         ps[i] = (01 & n) + '0'; // 01 是八进制书写方式, 前面加0
196     ps[size] = '\0';
197     return ps;
198 }
199
200 void show_bstr (const char * str) {
201     int i = 0;
202     while (str[i]) { // 把上面转化成二进制的字符串, 按4个一组分开来

```

```
203     putchar (str[i]);    // 输出二进制数字字符（0或1）
204     if (++i % 4 == 0 && str[i])
205         putchar(' ');
206 }
207 }
```